TECHNICAL ROADMAP & DELIVERY PLAN

EstiMate

Project Cost & Resource Estimation Platform

A web platform that converts a project description into a defensible cost estimate, a staffing plan, a schedule, and a progress-tracking model.

Document type	Engineering roadmap / build specification
Audience	Founders, engineering leads, investors, prospective collaborators
Build mode	Solo senior engineer, full-time (40 h/week) baseline
Baseline duration	MVP 13 weeks · Full v1.0 22 weeks
Status	Draft v1.0 — pre-development

Contents

1	Problem statement and product thesis	3
2	Scope: what v1 does and explicitly does not do	4
3	The estimation engine — the actual hard part	5
4	Worked example: a real estimate end to end	7
5	System architecture	8
6	Technology stack and justification	9
7	Data model	10
8	API surface	11
9	Repository structure	12
10	Phase-by-phase roadmap with effort estimates	13
11	Problems you will hit, and how to handle them	15
12	Non-functional requirements	17
13	Infrastructure cost model	17
14	Risk register	17
15	Definition of done and success metrics	18
16	Glossary	18

How to read this document. Sections 1–4 are the product and domain argument — read these if you want to understand *why* the product is credible. Sections 5–9 are the technical blueprint. Section 10 is the schedule. Section 11 is the section most roadmaps omit: the failure modes that are specific to estimation software, and the engineering decisions that avoid them.

1. Problem statement and product thesis

1.1 The problem

Software project estimates are wrong in a predictable direction. Across the industry, a large majority of projects overrun their original budget or schedule, and the overrun is rarely caused by bad engineering — it is caused by the estimate being produced as a single number, by one person, from intuition, with the assumptions left undocumented.

Three concrete failures repeat:

- **Point estimates masquerade as commitments.** "This will cost forty lakh" is stated with no confidence interval, so nobody can tell whether it means 35–45 or 25–90.
- **Effort is converted to schedule by division.** Sixty person-months becomes "six people, ten months", ignoring that staffing has a maximum useful rate and that adding people to a late project makes it later.
- **Assumptions evaporate.** The estimate assumed an existing auth system and no compliance requirement. Six weeks in, neither assumption holds, and there is no artefact anyone can point to.

1.2 The thesis

> EstiMate replaces the spreadsheet-and-intuition estimate with a structured, versioned, auditable model that outputs a **probability distribution**, not a number — plus the staffing curve, the schedule, and the assumption log that produced it.

1.3 Who it is for

Segment	Job to be done	What they need from v1
Services agencies 10–200 people	Produce a client proposal with a defensible price in under a day	Fast intake, branded PDF export, margin controls, rate cards
Startup founders / CTOs	Decide whether a feature is affordable before committing	Quick-estimate mode, scenario comparison, runway impact
Enterprise PMO	Standardise estimates across teams and defend them in budget review	SSO, audit trail, org-wide rate cards, calibration on historical data
Freelancers	Stop under-quoting	Single-user free tier, simple output

1.4 Why this is defensible

The moat is not the UI — it is **calibration**. Every project an organisation completes inside the tool produces an actual-versus-estimated data point. After twenty projects the model is tuned to that organisation's real productivity, and the estimates become materially better than any generic calculator. That data asset is why customers stay, and it is why the historical-data capture (Section 3.7) must exist in v1 even though it produces no immediate value.

2. Scope: what v1 does and explicitly does not do

2.1 In scope for MVP (Phases 0–6)

Capability	Definition of the feature
Project intake	Guided wizard: project type, module/feature breakdown, complexity per module, tech stack, team seniority mix, constraints. Quick mode (8 fields) and detailed mode (40+ fields).
Size estimation	Function-point-style sizing derived from the feature breakdown, with manual override.
Effort estimation	COCOMO II Early Design model plus a three-point (PERT) path; both run, results reconciled and shown side by side.
Uncertainty modelling	Monte Carlo simulation, 10,000 iterations, producing P50 / P80 / P95 cost and duration.
Staffing plan	Role-by-role headcount over time (Rayleigh-shaped staffing curve), not a flat average.
Schedule generation	WBS with dependencies, critical path, calendar-aware dates honouring holidays and part-time allocation.
Cost model	Rate cards per role / seniority / geography, multi-currency, overhead and margin, contingency reserve.
Assumption log	Every assumption captured as a first-class record attached to the estimate version.
Reporting	Branded PDF proposal and XLSX export of the full breakdown.
Versioning	Estimates are immutable snapshots; re-running creates a new version with a diff view.

2.2 In scope for v1.0 (Phases 7–10)

Capability	Definition of the feature
Scenario / what-if	Clone an estimate, change inputs, compare 2–4 scenarios on one screen.
Progress tracking	Earned Value Management: planned value, earned value, actual cost, SPI, CPI, estimate-at-completion. Answers "what percentage is done, and are we on track".
Calibration	Capture actuals on closed projects; derive an organisation productivity multiplier applied to future estimates.
Multi-tenancy & RBAC	Organisations, workspaces, roles (owner / estimator / viewer), invitations.
Billing	Subscription tiers via Stripe, seat-based.

2.3 Explicitly out of scope

> **This is the single most important page in the document.** Estimation tools die by turning into bad project-management tools. Every item below has been considered and deliberately rejected for v1.

- **Task execution / kanban boards.** You are not competing with Jira. Integrate with it later, do not rebuild it.
- **Timesheets.** Import actuals from Jira, Harvest, or Toggl. Do not build time entry.
- **Invoicing and accounting.** Export to the customer's system.
- **Resource hiring / recruitment workflows.**
- **Gantt editing by drag-and-drop.** Render the schedule read-only in v1; editable Gantt is a multi-week feature with deep undo/redo and conflict semantics.
- **AI free-text estimation as the primary path.** An LLM can assist with the feature breakdown (Phase 7+), but the number must come from a deterministic, explainable model. If a CFO asks "why 62 lakh", "the model said so" ends the sale.

3. The estimation engine — the actual hard part

Ninety percent of the engineering risk in this product lives in this section. The CRUD around it is routine. Build the engine as a pure, side-effect-free library with no database access, so it can be unit-tested exhaustively and versioned independently.

3.1 Pipeline

```
INPUT (project profile)
   |
   v
[1] SIZING          feature breakdown  ->  size in function points / story points
   |
   v
[2] EFFORT          size + cost drivers ->  effort in person-months (nominal)
   |
   v
[3] CALIBRATION     x organisation productivity multiplier (learned from history)
   |
   v
[4] UNCERTAINTY     Monte Carlo over input distributions -> effort distribution
   |
   v
[5] SCHEDULE        effort -> minimum duration, staffing curve, WBS, critical path
   |
   v
[6] COST            staffing curve x rate cards x calendar -> cost distribution
   |
   v
[7] RESERVES        + contingency (P80 - P50) + management reserve + margin
   |
   v
OUTPUT (estimate: P50 / P80 / P95 cost, duration, staffing plan, assumptions)
```

3.2 Stage 1 — Sizing

The user does not know how many lines of code their project is, and should never be asked. They know their features. Size is therefore derived from a structured feature breakdown, where each module carries a type and a complexity rating.

Element type	Low	Average	High	Example
External input (form, write API)	3	4	6	Create-invoice form
External output (report, export)	4	5	7	Monthly PDF statement
External inquiry (search, lookup)	3	4	6	Customer search
Internal logical file (entity)	7	10	15	Orders table + logic
External interface file (3rd-party)	5	7	10	Stripe integration

Unadjusted function points are the weighted sum. Convert to equivalent source statements using a language/stack productivity factor (backing table maintained per stack), or keep working in function points throughout — the latter is cleaner and avoids fake precision. Provide a manual override for teams that already size in story points.

3.3 Stage 2 — Effort (COCOMO II, Early Design)

```
Effort(PM) = A * Size^E * PROD(EM_i)

  A     = 2.94                    (calibration constant)
  Size  = KSLOC  (or FP-equivalent)
  E     = B + 0.01 * SUM(SF_j)    B = 0.91
  SF_j  = 5 scale factors, each 0 (very high) .. 5+ (very low)
  EM_i  = 7 early-design effort multipliers
```

Scale factor	What it captures	Range
PREC — Precedentedness	Has the team built something like this before	0.00 – 6.20
FLEX — Development flexibility	How rigid the requirements and process are	0.00 – 5.07
RESL — Architecture / risk resolution	How much design and risk work is done up front	0.00 – 7.07
TEAM — Team cohesion	Stakeholder alignment, co-location, trust	0.00 – 5.48
PMAT — Process maturity	CI/CD, review discipline, testing culture	0.00 – 7.80

Effort multiplier	What it captures	Typical range
RCPX — Product reliability & complexity	Failure consequence, algorithmic difficulty	0.49 – 1.91
RUSE — Required reusability	Build-for-reuse overhead	0.95 – 1.24

PDIF — Platform difficulty	Timing, memory, volatility of platform	0.87 – 2.61
PERS — Personnel capability	Analyst and programmer capability	0.50 – 2.12
PREX — Personnel experience	Domain, language, and tooling familiarity	0.62 – 1.59
FCIL — Facilities / tooling	Tooling quality, environment friction	0.62 – 1.43
SCED — Required schedule compression	Penalty for compressing below nominal	1.00 – 1.43

> **Note on PERS.** The span between a weak and a strong team is roughly 4x. This is real and well documented, but it is commercially awkward — no client wants to see "we rated your team below average". Expose it as a neutral experience questionnaire, never as a capability score.

3.4 Stage 4 — Uncertainty (Monte Carlo)

Every uncertain input is a distribution, not a number:

- Module complexity → triangular distribution (optimistic, likely, pessimistic).
- Productivity multiplier → lognormal, fitted to the organisation's historical spread.
- Scope growth → lognormal with a mean of 1.15–1.35 depending on requirement stability. **Scope creep must be modelled explicitly.** It is the largest single driver of overrun and pretending it is zero is why estimates fail.
- Attrition and unplanned absence → Bernoulli events with effort penalties.

Run 10,000 iterations, sort the results, report percentiles. The headline number shown to the user is **P80**, not P50 — a P50 estimate is a coin flip and should never be quoted to a client.

3.5 Stage 5 — Schedule and staffing

```
Nominal duration:   TDEV = 3.67 * Effort^F,   F = 0.28 + 0.2 * (E - 0.91)
Average staffing:   N_avg = Effort / TDEV
Compression floor:  TDEV cannot be compressed below ~0.75 * TDEV_nominal at any cost
Communication tax:  overhead ~ k * n * (n - 1) / 2  applied per team above ~8 people
```

Staffing is not flat. Model it as a Rayleigh curve: ramp up, peak at roughly 40% of elapsed time, taper during hardening. Then decompose by role — backend, frontend, QA, DevOps, design, PM — using stack-specific role-mix ratios, because a data-heavy platform and a marketing site have very different role mixes at the same size.

Split the effort across the lifecycle using phase ratios (indicative for a mid-size product):

Phase	Requirements	Design	Implementation	Test & integration	Transition
Share of effort	7%	17%	48%	22%	6%
Share of schedule	12%	21%	40%	20%	7%

3.6 Stage 7 — Reserves and price

```
Contingency reserve = P80_cost - P50_cost        (owned by the project, for known-unknowns)
Management reserve  = 5-10% of P80_cost         (owned by the sponsor, for unknown-unknowns)
Internal cost       = P80_cost + management reserve
Client price        = internal cost / (1 - target margin)
```

3.7 Calibration loop (build the capture in v1, the learning in v1.1)

On project close, capture actual effort, actual duration, actual cost, and final delivered size. Compute the ratio of actual to estimated for each closed project, fit a multiplier with a credibility weight based on sample count, and apply it to new estimates for that organisation. Show the user their own calibration curve — it is the most persuasive screen in the product.

4. Worked example: a real estimate end to end

Scenario: a B2B logistics dashboard. Web app, Postgres backend, three third-party integrations, moderate reliability requirement, experienced team, requirements moderately stable.

Step	Calculation	Result
1. Feature breakdown	34 inputs, 12 outputs, 18 inquiries, 16 entities, 3 interfaces	~420 unadjusted FP
2. Complexity adjustment	420 x 0.95	400 FP
3. Convert to KSLOC	400 FP x 45 SLOC/FP (TypeScript)	18.0 KSLOC
4. Scale exponent	E = 0.91 + 0.01 x (SUM SF = 18.7)	E = 1.097
5. Nominal effort	$2.94 \times 18.0^{1.097} = 2.94 \times 23.9$	70.3 person-months
6. Effort multipliers	x 0.85 (strong team, good tooling, nominal schedule)	59.8 person-months
7. Org calibration	x 1.00 (no history yet — defaults to industry)	59.8 PM
8. Monte Carlo (10k runs)	incl. scope growth mean 1.22, sigma 0.28	P50 **61 PM** · P80 **78 PM** · P95 **94 PM**
9. Nominal duration	$3.67 \times 61^{0.317} = 3.67 \times 3.67$	13.5 months
10. Peak / average staffing	61 / 13.5 avg, Rayleigh peak ~1.6x	4.5 avg FTE, ~7 peak FTE
11. Role split	3 BE, 2 FE, 1 QA, 0.5 DevOps, 0.5 Design, 0.5 PM at peak	7.5 peak heads
12. Cost at blended INR 1,80,000 / FTE-month loaded	78 PM x 1.80L	**INR 1.40 Cr (P80)**
13. Management reserve	+ 7%	INR 1.50 Cr internal cost
14. Client price at 30% margin	1.50 / 0.70	**INR 2.14 Cr**

> **What the user sees.** Not step 14 alone. They see: "80% confidence this delivers in 13–16 months for INR 1.35–1.55 Cr, peaking at 7 people in month 6, assuming requirements are frozen at month 2 and the three integrations have sandbox access from day one." Those two assumptions are clickable and versioned. **That output is the product.**

4.1 The progress question

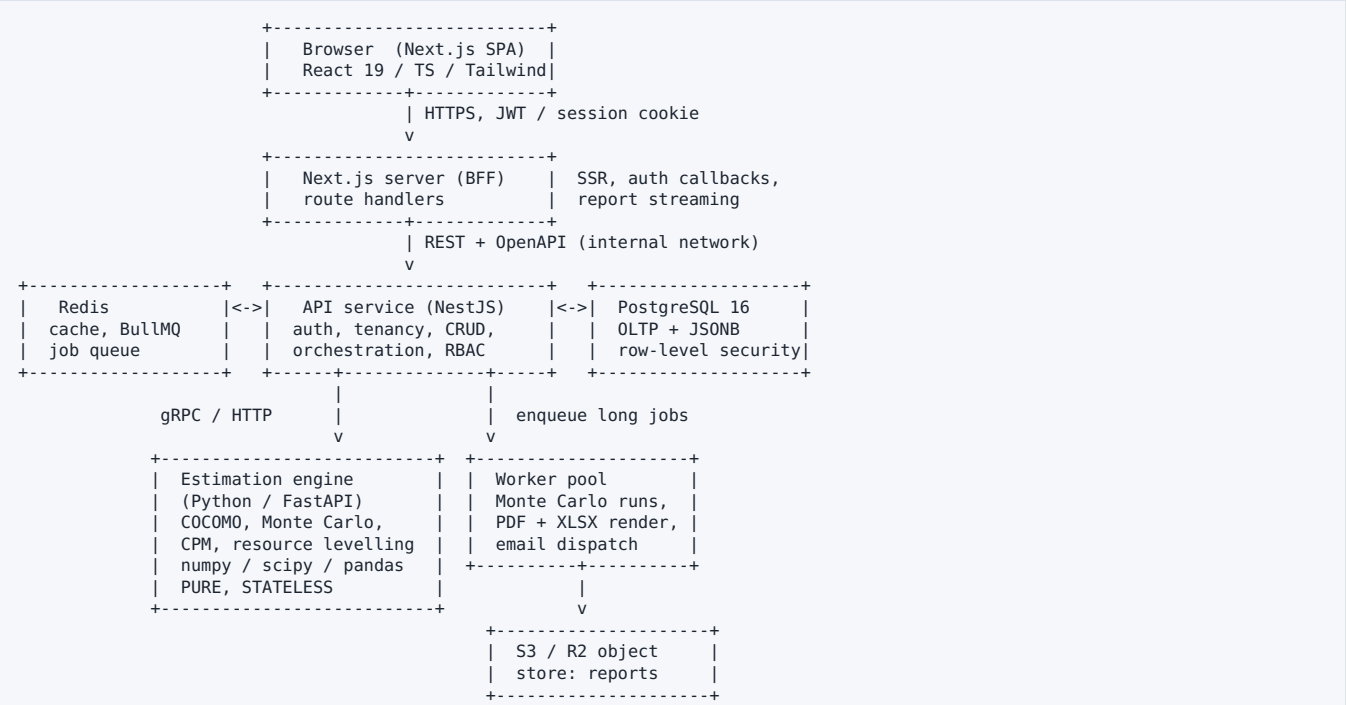
The brief asked for "how much percentage of work is done in how many days". That is Earned Value Management, and it is Phase 7:

```
PV  Planned Value    = budgeted cost of work scheduled to date
EV  Earned Value     = budgeted cost of work actually completed  (this is "% done" in money)
AC  Actual Cost      = money actually spent to date

SPI = EV / PV     schedule performance index   (< 1.0 = behind)
CPI = EV / AC     cost performance index       (< 1.0 = over budget)
EAC = BAC / CPI   forecast cost at completion
ETC = EAC - AC    forecast remaining spend
```

Percent complete is EV / BAC. Critically, it is weighted by budgeted effort, not by task count — "18 of 30 tasks done" is meaningless when the remaining 12 hold 70% of the work.

5. System architecture



5.1 Why the engine is a separate service

- **Language fit.** numpy vectorises a 10,000-iteration Monte Carlo over a 400-node WBS in milliseconds. The same loop in Node is one to two orders of magnitude slower and far uglier.
- **Isolation of the risky part.** The engine is the component most likely to change. Versioning it independently lets you pin an estimate to `engine@2.3.1` forever, which is mandatory for audit.
- **Horizontal scale.** CPU-bound work scales separately from the I/O-bound API.
- **Testability.** A pure service with no DB is trivially property-tested and golden-file tested.

> **Counter-argument, stated honestly.** Two languages means two toolchains, two CI pipelines, and duplicated type definitions. If you are solo and want to ship faster, build the engine as a separate *package* in TypeScript inside the monorepo, keep the interface boundary clean, and extract it into a Python service only when Monte Carlo latency becomes the bottleneck. Both are defensible; the interface discipline matters more than the language.

5.2 Synchronous vs asynchronous

Operation	Mode	Target latency
Quick estimate (deterministic, no simulation)	Synchronous	< 400 ms p95
Full estimate with 10k Monte Carlo	Sync if < 2 s, else queued	< 2 s typical
Resource levelling over a large WBS	Queued job, progress via SSE	< 30 s
PDF / XLSX report generation	Queued job, notify on completion	< 15 s
Scenario comparison (4 scenarios)	Queued, parallel fan-out	< 20 s

6. Technology stack and justification

6.1 Recommended stack

Layer	Choice	Why	Alternative considered
Frontend	Next.js 15 (App Router), React 19, TypeScript strict	SSR for marketing pages, server actions for forms, one deployment target	Vite + React Router — faster dev loop, no SSR
Styling	Tailwind CSS + shadcn/ui	Own the component source; no fighting a design system you cannot edit	MUI — faster start, harder to differentiate
State / data	TanStack Query + Zustand	Server cache and UI state are different problems; do not use Redux for both	RTK Query
Forms	react-hook-form + Zod	The intake wizard is 40+ fields with conditional branches; uncontrolled forms are the only sane option	Formik — heavier re-renders
Charts	Recharts for standard, visx/D3 for the Gantt and staffing curve	Gantt and S-curves need custom rendering	Chart.js, Highcharts (licensed)
API	NestJS + TypeScript	Opinionated DI, modules, guards, interceptors, OpenAPI generation; scales with team size	Express (too unopinionated), Fastify, Go
Engine	Python 3.12 + FastAPI + numpy / scipy / pandas	Vectorised simulation, mature statistics, readable numeric code	TypeScript package (see 5.1)
Database	PostgreSQL 16	Transactions, JSONB for flexible estimate payloads, row-level security for tenancy, window functions for EVM	MongoDB — rejected: this is relational, financial, and needs transactions
ORM	Prisma (Node) + SQLAlchemy 2 (Python, read-only)	Typed client, first-class migrations	Drizzle, TypeORM
Cache / queue	Redis 7 + BullMQ	One dependency covers cache, rate limiting, and jobs	RabbitMQ, SQS
Auth	Auth.js (MVP) → Keycloak or WorkOS (enterprise SSO)	Email + OAuth now; SAML/OIDC is an enterprise gate you will hit	Clerk — fastest, but per-MAU pricing
Reports	Playwright headless Chromium → PDF; ExcelJS → XLSX	HTML templates mean one rendering language for screen and print	ReportLab, Puppeteer, LaTeX
Payments	Stripe Billing	Seat-based subscriptions, tax handling, hosted portal	Razorpay/Paddle for India-first billing
Infra (MVP)	Railway or Render + Neon Postgres + Upstash Redis	Deploy in hours, not weeks; pay near zero pre-revenue	AWS from day one — premature
Infra (scale)	AWS ECS Fargate + RDS + ElastiCache + S3, Terraform	Enterprise buyers ask about region, VPC, and backups	Kubernetes — do not, while solo
CI/CD	GitHub Actions + Turborepo remote cache	Monorepo-aware, affected-only builds	CircleCI, GitLab CI
Observability	Sentry + OpenTelemetry + Axiom or Grafana Cloud	Errors, traces, and logs correlated by trace ID	Datadog — excellent, expensive
Testing	Vitest, Playwright, pytest, Hypothesis, fast-check, Testcontainers	Property-based testing is essential for the engine	Jest, Cypress

6.2 Local requirements

```
Node        >= 22 LTS          pnpm >= 9      (workspaces + Turborepo)
Python      >= 3.12             uv or Poetry
Docker      >= 24               docker compose v2
Postgres    16   (container)    Redis 7 (container)
Editor      VS Code + ESLint, Prettier, Prisma, Ruff, Pylance
RAM         16 GB minimum       (Playwright + Postgres + Redis + two dev servers)
```

6.3 Version-pinning discipline

Pin exact versions in the engine service and record the engine version on every stored estimate. An estimate produced in March must reproduce byte-identically in December, or the audit story collapses. Seed every random number generator and store the seed on the estimate record.

7. Data model

7.1 Core entities

Entity	Key fields	Notes
Organization	id, name, slug, currency_default, locale, plan, calibration_factor	Tenancy root
User	id, email, name, password_hash / sso_subject, mfa_secret	
Membership	org_id, user_id, role [owner, admin, estimator, viewer]	Users can belong to many orgs
Project	id, org_id, name, client_name, type, status, owner_id	Container, mutable
EstimateVersion	id, project_id, version_no, engine_version, rng_seed, inputs_json, created_by, locked_at	**Immutable once locked**
Module	estimate_version_id, name, element_type, complexity, count, manual_fp_override	The feature breakdown
CostDriver	estimate_version_id, code (PERS, PREX, RCPX…), rating, multiplier_value	Snapshotted values, not references
RateCard / RateCardEntry	org_id, name, currency, effective_from · role, seniority, region, rate_minor_units, unit	Money in integer minor units
EstimateResult	estimate_version_id, p50/p80/p95 effort, duration, cost_minor, distribution_json, computed_at	Cached engine output
Assumption	estimate_version_id, text, category, impact_if_false, owner, status	First-class, exported into the PDF
RiskItem	estimate_version_id, description, probability, impact_effort_pm, mitigation	Feeds the Monte Carlo
WbsTask	estimate_version_id, wbs_code, name, effort_pm, role, earliest_start, latest_finish, is_critical	Generated, not hand-entered in v1
Dependency	predecessor_id, successor_id, type [FS, SS, FF, SF], lag_days	Must be acyclic — validate
Allocation	wbs_task_id, role, headcount, period_start, period_end, allocation_pct	The staffing plan
CapacityCalendar	org_id, region, working_days, holidays[], default_hours_per_day	Holiday-aware scheduling
ProgressSnapshot	project_id, as_of_date, pv, ev, ac, spi, cpi, eac	EVM time series
ActualsRecord	project_id, actual_effort_pm, actual_cost_minor, actual_duration_days, delivered_fp	Feeds calibration
Scenario	project_id, base_version_id, label, overrides_json	What-if branches
AuditLog	org_id, actor_id, action, entity, before_json, after_json, ip, at	Append-only, enterprise requirement

7.2 Three data rules that are non-negotiable

1. **Money is never a float.** Store integer minor units plus an ISO-4217 currency code. `0.1 + 0.2` producing `0.30000000000000004` in a quotation is a credibility-ending bug. Use `numeric` in Postgres, `Decimal` in Python, integer paise/cents in JS.
2. **Estimates are append-only.** Never UPDATE a locked EstimateVersion. Re-running creates version n+1. Enterprise buyers will ask what changed between the March quote and the June quote; a diff view answers that in one screen.
3. **Every query is scoped by org_id.** Enforce it at the database layer with Postgres row-level security, not only in application code. Application-layer-only tenancy is how IDOR vulnerabilities ship.

8. API surface

REST with an OpenAPI 3.1 contract generated from NestJS decorators. Cursor pagination. Idempotency keys on all POSTs that create financial records. Versioned under `/v1` from day one.

```
AUTH
POST   /v1/auth/register                    POST /v1/auth/login
POST   /v1/auth/refresh                     POST /v1/auth/logout
GET    /v1/auth/sso/:org/saml               (enterprise, Phase 8)

ORGANISATION
GET    /v1/orgs/:id                         PATCH /v1/orgs/:id
POST   /v1/orgs/:id/invitations             GET   /v1/orgs/:id/members
GET    /v1/orgs/:id/rate-cards              POST  /v1/orgs/:id/rate-cards
GET    /v1/orgs/:id/calibration             (actual vs estimated curve)

PROJECTS
GET    /v1/projects?status=&cursor=         POST  /v1/projects
GET    /v1/projects/:id                     PATCH /v1/projects/:id
DELETE /v1/projects/:id                     (soft delete)

ESTIMATES
POST   /v1/projects/:id/estimates           create draft version
GET    /v1/projects/:id/estimates           list versions
GET    /v1/estimates/:vid                   full payload
PATCH  /v1/estimates/:vid                   only while unlocked
POST   /v1/estimates/:vid/modules           feature breakdown rows
POST   /v1/estimates/:vid/compute           -> 202 + job_id  (or 200 if fast path)
POST   /v1/estimates/:vid/lock              make immutable
GET    /v1/estimates/:vid/diff/:otherVid    version comparison

ENGINE  (internal only, not public)
POST   /internal/engine/size                FP calculation
POST   /internal/engine/effort              COCOMO II
POST   /internal/engine/simulate            Monte Carlo
POST   /internal/engine/schedule            CPM + resource levelling
GET    /internal/engine/version             pinned version string

SCENARIOS + PROGRESS
POST   /v1/estimates/:vid/scenarios         GET /v1/scenarios/compare?ids=
POST   /v1/projects/:id/progress            submit actuals for a date
GET    /v1/projects/:id/evm                 PV/EV/AC/SPI/CPI/EAC series

JOBS + EXPORTS
GET    /v1/jobs/:jobId                      status polling
GET    /v1/jobs/:jobId/events               SSE progress stream
POST   /v1/estimates/:vid/exports           { format: pdf | xlsx | csv }
GET    /v1/exports/:exportId/download        signed URL, 15-min TTL
```

8.1 API conventions

- Errors follow RFC 9457 *problem+json*: `type`, `title`, `status`, `detail`, `instance`.
- Rate limit: 100 req/min per user, 10/min on `/compute`. Return 429 with `Retry-After`.
- All monetary values serialise as `{ "amount_minor": 21400000, "currency": "INR" }` — never a bare number.
- Long operations return `202 Accepted` with a `Location` header pointing at the job resource.

9. Repository structure

```
estimate/
├── apps/
│   ├── web/                        Next.js 15 frontend
│   │   ├── src/app/
│   │   │   ├── (marketing)/        public pages, SSG
│   │   │   ├── (auth)/             login, register, invite accept
│   │   │   └── (app)/
│   │   │       ├── projects/
│   │   │       ├── estimates/[id]/
│   │   │       │   ├── intake/     the wizard
│   │   │       │   ├── results/    distributions, staffing, schedule
│   │   │       │   ├── scenarios/
│   │   │       │   └── assumptions/
│   │   │       ├── progress/       EVM dashboard
│   │   │       └── settings/       rate cards, members, calendars
│   │   ├── src/components/
│   │   │   ├── ui/                 shadcn primitives
│   │   │   ├── charts/             gantt, s-curve, tornado, histogram
│   │   │   └── wizard/             step components + schema
│   │   ├── src/lib/                api client, formatters, currency
│   │   └── e2e/                    Playwright specs
│   │
│   └── api/                        NestJS backend
│       ├── src/modules/
│       │   ├── auth/               strategies, guards, refresh rotation
│       │   ├── orgs/               tenancy, members, invitations
│       │   ├── projects/
│       │   ├── estimates/          versioning, locking, diffing
│       │   ├── rate-cards/
│       │   ├── scenarios/
│       │   ├── progress/           EVM aggregation
│       │   ├── exports/            pdf + xlsx producers
│       │   ├── jobs/               BullMQ queues, processors, SSE
│       │   └── engine-client/      typed HTTP client for the engine
│       ├── src/common/             filters, interceptors, decorators, RLS ctx
│       ├── prisma/                 schema.prisma, migrations/, seed.ts
│       └── test/                   integration tests w/ Testcontainers
│
├── services/
│   └── engine/                     Python estimation engine
│       ├── app/
│       │   ├── api/                FastAPI routers
│       │   ├── domain/
│       │   │   ├── sizing.py              function point model
│       │   │   ├── cocomo.py              effort model + driver tables
│       │   │   ├── montecarlo.py          simulation, seeded RNG
│       │   │   ├── scheduling.py          CPM, forward/backward pass
│       │   │   ├── levelling.py           resource levelling heuristic
│       │   │   ├── staffing.py            Rayleigh curve, role mix
│       │   │   ├── costing.py             rate cards, Decimal money
│       │   │   └── evm.py                 PV/EV/AC/SPI/CPI/EAC
│       │   ├── data/               driver tables, productivity factors (YAML)
│       │   └── schemas/            pydantic request/response models
│       └── tests/
│           ├── unit/               per-function
│           ├── property/           Hypothesis invariants
│           └── golden/             frozen input->output fixtures
│
├── packages/
│   ├── shared-types/               zod schemas -> TS types, generated from OpenAPI
│   ├── ui/                         cross-app components
│   ├── config/                     eslint, tsconfig, tailwind presets
│   └── report-templates/           HTML/CSS templates for PDF rendering
│
├── infra/
│   ├── docker/                     Dockerfiles per app
│   ├── compose.dev.yml             postgres + redis + services
│   ├── terraform/                  AWS (Phase 10)
│   └── k8s/                        deferred, do not use while solo
│
├── docs/
│   ├── adr/                        architecture decision records
│   ├── engine-methodology.md       the model, cited and explained
│   ├── api.md                      generated from OpenAPI
│   └── runbook.md                  on-call procedures
│
├── .github/workflows/              ci.yml, deploy.yml, engine-ci.yml
├── turbo.json   pnpm-workspace.yaml   .env.example   README.md
```

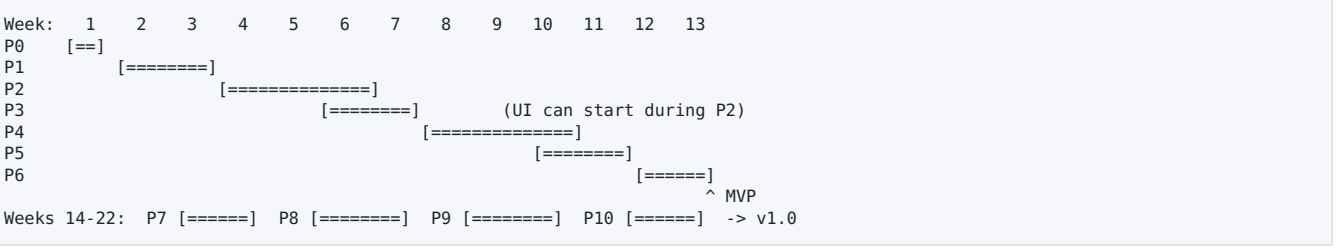
> **One structural rule.** `services/engine/app/domain/` must have zero imports from `api/`, zero database access, and zero I/O. Pure functions in, pure data out. This is what makes the most valuable part of the codebase testable, portable, and replaceable.

10. Phase-by-phase roadmap with effort estimates

Baseline: one experienced full-stack engineer, 40 productive hours per week. Multiply elapsed weeks by ~2.5 for evenings-and-weekends work (12–15 h/week). A team of three compresses this to roughly 10–12 weeks, not 7 — see Brooks's law, Section 11.3.

#	Phase	Deliverables	Hours	Weeks
0	**Discovery & spec**	Competitor teardown (5 tools), 8 customer interviews, methodology decision documented, wireframes for 6 core screens, data model on paper, ADRs 1–5	40	1.0
1	**Foundations**	Monorepo + Turborepo, Docker compose, Prisma schema v1 + migrations, NestJS skeleton, Next.js skeleton, auth (register/login/refresh/reset), org + membership + RBAC guards, RLS policies, GitHub Actions CI, Sentry, seed script	80	2.0
2	**Estimation engine core** Highest risk	FastAPI service, sizing module, COCOMO II with full driver tables, Monte Carlo with seeded RNG, unit + property + golden tests, engine versioning, benchmark suite (10k iterations under 2 s)	120	3.0
3	**Intake & estimate CRUD**	Multi-step wizard with Zod schemas and draft autosave, module breakdown editor, cost-driver questionnaire, rate-card CRUD, estimate versioning + locking, engine-client integration, compute endpoint	80	2.0
4	**Scheduling & staffing** Second highest risk	WBS generation from modules, dependency graph + cycle detection, CPM forward/backward pass, resource-levelling heuristic, Rayleigh staffing curve, role-mix decomposition, capacity calendars with holidays, BullMQ job pipeline + SSE progress	120	3.0
5	**Results dashboard**	Cost/duration histogram with P50/P80/P95 markers, tornado sensitivity chart, staffing-curve area chart, read-only Gantt with critical path highlighted, cost breakdown by role and phase, assumptions and risk panels	80	2.0
6	**Reporting & export**	HTML report templates, Playwright PDF pipeline with org branding, ExcelJS multi-sheet workbook, CSV export, signed download URLs, email delivery	60	1.5
MVP SUBTOTAL — demonstrable, sellable to design partners			**480**	**12.5**
7	**Scenarios & progress**	Scenario cloning + override model, side-by-side comparison of up to 4, actuals ingestion, EVM computation and time series, S-curve dashboard, variance alerts	60	1.5
8	**Tenancy, billing, enterprise**	Invitation flow, granular RBAC, audit log, SAML/OIDC SSO, Stripe subscriptions + webhooks + seat metering, plan gating, org settings	80	2.0
9	**Hardening**	Test coverage to 80% on engine and API, load test to 200 concurrent estimates, N+1 query elimination, OWASP Top 10 pass, dependency audit, accessibility pass to WCAG 2.2 AA, i18n scaffolding, error-state and empty-state design	80	2.0
10	**Launch**	Terraform IaC, staging + production environments, blue-green deploy, backup and restore drill, runbook, methodology whitepaper, onboarding tour, marketing site, beta with 5 design partners	60	1.5
v1.0 TOTAL			**760**	**~22**

10.1 Critical path and parallelism



Phase 3's UI work can overlap Phase 2 by roughly a week if you define the engine's request and response contract first and mock it. Defining that contract in week 3 is the highest-leverage hour in the entire schedule.

10.2 Milestones and gates

Week	Milestone	Gate criterion — do not proceed until true
1	Spec signed off	Data model and methodology written down; 8 interviews done
3	Walking skeleton	A user can register, create an org, and see an empty project list in production

6	Engine correct	Golden fixtures pass; a hand-worked example matches the engine to 2 decimal places
8	First end-to-end estimate	Wizard input produces a stored, locked, versioned estimate
11	Schedule & staffing live	Gantt and staffing curve render from real engine output
13	**MVP / design-partner ready**	Three external users complete an estimate unaided and export a PDF
17	Commercial ready	A stranger can sign up, pay, and produce a quote
22	**v1.0 launch**	Load-tested, backed up, documented, monitored, restore drill passed

11. Problems you will hit, and how to handle them

Ordered by expected cost of getting it wrong. The severity tag reflects impact, not likelihood.

11.1 Cold start: no historical data Severity: critical

Problem. The model needs calibration data you do not have. Generic COCOMO coefficients derived from 1990s projects can be off by 2–3x for a modern TypeScript team using managed services.

Handling. Ship with published industry baselines as the default and label them as such in the UI. Make calibration a visible first-class feature from day one: "Your estimates improve after 3 completed projects." Offer a back-fill flow where an organisation enters 5 past projects (estimated vs actual) during onboarding — this both calibrates the model and is the single strongest activation event in the funnel. Never hide the fact that an uncalibrated estimate is wide.

11.2 Garbage in, garbage out — and it will be your fault Severity: critical

Problem. A user gives shallow inputs, gets a number, quotes a client, overruns by 60%, and blames the tool.

Handling. Four defences, all in v1. (a) Never display a single number without its interval; the UI should make a bare point estimate literally impossible to screenshot. (b) Compute an input-quality score and surface it prominently: "12 of 40 fields answered — confidence: low". (c) Force at least three assumptions before an estimate can be locked. (d) Put the assumptions and the confidence interval on page one of the exported PDF, not in an appendix. Add clear terms stating that estimates are forecasts, not guarantees.

11.3 Effort divided by people does not equal time Severity: critical

Problem. The naive implementation lets a user set "60 person-months, 60 people, 1 month". Every experienced buyer who sees that output stops trusting the product immediately.

Handling. Enforce a compression floor: schedule cannot go below roughly 75% of nominal duration regardless of headcount, and compression below nominal costs extra effort (the SCED multiplier, up to 1.43x). Model communication overhead as growing with n(n-1)/2 above a team size of about eight. Model a ramp-up penalty: a new joiner is roughly 0% productive in week 1, 50% by week 4, and consumes about 25% of an existing engineer's time during onboarding. When a user tries to compress, show the cost curve instead of an error message — the teaching moment is the product's best demo.

11.4 Floating-point money Severity: high

Problem. A quotation that reads INR 21,40,000.0000000003, or totals that differ by one paisa between the dashboard and the PDF.

Handling. Integer minor units everywhere in storage and transport; `numeric` in Postgres; `Decimal` with an explicit context in Python; never `float` in a money path. Round once, at the presentation boundary, using banker's rounding. Write a lint rule banning `number` on any field whose name matches `/cost|price|rate|amount|total/`.

11.5 Calendars, holidays, and timezones Severity: high

Problem. "12 weeks" is meaningless without a calendar. Diwali, Christmas, a four-day week, a person at 60% allocation, a team split across India and Germany with different holiday sets — each shifts the end date, and users notice immediately when dates are wrong.

Handling. Build a capacity-calendar model in Phase 4, not later; retrofitting it means rewriting the scheduler. Store all instants in UTC, render in the org's timezone. Use a maintained library (date-fns-tz, python `zoneinfo`), never hand-rolled date arithmetic. Let each region carry its own holiday set and allow per-resource overrides for leave.

11.6 Resource levelling is NP-hard Severity: high

Problem. Assigning limited people to a dependency graph while minimising duration is combinatorially explosive. A naive solver will hang on a 300-task WBS.

Handling. Do not seek optimality. Use a priority-rule heuristic — serial schedule generation with minimum-slack-first ordering — which is near-optimal in practice and runs in near-linear time. Cap iterations and always return the best solution found so far. Run it as a queued job with a hard timeout. Detect dependency cycles before scheduling and reject with a clear message naming the cycle.

11.7 Monte Carlo performance Severity: high

Problem. 10,000 iterations over 400 tasks in a naive Python loop takes minutes and blocks a worker.

Handling. Vectorise with numpy — sample all iterations for all tasks as a single (10000 x 400) array operation. This turns minutes into tens of milliseconds. Cache results keyed by a hash of the normalised inputs plus the engine version. Downgrade to 2,000 iterations for the live preview and run the full 10,000 only on explicit compute. Never run simulation in the request thread above a size threshold.

11.8 Reproducibility and audit Severity: high

Problem. A client asks in December why the March estimate differed. Meanwhile you have shipped four engine changes and the March estimate no longer reproduces.

Handling. Store on every estimate: the full input payload, the engine semantic version, the driver-table version, and the RNG seed.

Locked estimates are immutable. Engine changes are additive and versioned; old versions remain callable. Build the version diff view in Phase 3 — it is cheap then and expensive later.

11.9 Multi-tenant data leakage Severity: high

Problem. One missing `WHERE org_id = ?` exposes a competitor's pricing. For this product specifically, that is an existential incident.

Handling. Postgres row-level security with a session variable set per request, so the database refuses cross-tenant reads even if application code is wrong. Use opaque UUIDv7 identifiers, never sequential integers. Write an automated test suite that, for every endpoint, attempts access as a member of a different org and asserts 404 (not 403 — do not leak existence).

11.10 Wizard abandonment Severity: high

Problem. The detailed model wants 40 inputs. Users abandon at input 12. Zero estimates, zero activation, no product.

Handling. Two paths from the first screen: *Quick estimate* (8 fields, 60 seconds, wide interval) and *Detailed estimate* (full model, narrow interval). Show the interval narrowing live as fields are answered — make the reward for more input visible and immediate. Prefill everything with type-specific defaults so no field is ever blank. Autosave drafts on every change. Offer templates: "E-commerce platform", "Internal admin tool", "Mobile app with backend".

11.11 Scope creep into a project-management tool Severity: medium, near-certain

Problem. Every design partner will ask for task assignment, then comments, then time tracking. Each request is reasonable. Collectively they turn a focused product into a worse Jira and add a year.

Handling. Write Section 2.3 into the README and revisit it at every planning session. The answer to execution features is an integration, not a rebuild: pull the WBS into Jira, pull actuals back for EVM. Say it out loud to design partners — "we estimate, we do not execute" — and most will agree.

11.12 Report rendering fidelity Severity: medium

Problem. PDFs with broken page breaks, missing fonts on the server, charts that render at the wrong size, or a table split awkwardly across two pages. Clients judge the product by this artefact.

Handling. Headless Chromium via Playwright with a print stylesheet, fonts bundled into the Docker image, and explicit `page-break-inside: avoid` rules. Render charts server-side to static SVG rather than screenshotting a live canvas. Keep a visual-regression test that diffs the generated PDF against golden images on every CI run.

11.13 Enterprise requirements arriving late Severity: medium

Problem. The first enterprise deal stalls on SSO, audit logs, data residency, a security questionnaire, and a DPA. Retrofitting audit logging across a mature codebase takes weeks.

Handling. Build the seams in Phase 1 even if unimplemented: an audit-log table written through a single interceptor, an abstract auth provider interface, region as a first-class config value, and configurable data-retention policies. Keep a security questionnaire answer bank from the first conversation onward.

11.14 Testing a probabilistic system Severity: medium

Problem. Assertions like `expect(cost).toBe(1400000)` fail randomly when the engine involves sampling.

Handling. Seed every RNG and assert exact values in golden tests. Separately, assert *invariants* with property-based testing: P50 ≤ P80 ≤ P95 always; increasing a module's complexity never decreases cost; doubling every module increases effort by more than 2x (diseconomy of scale); duration is always positive; the critical path length equals the project duration. These invariants catch real bugs that example-based tests miss.

11.15 Currency, FX, and multi-year escalation Severity: medium

Problem. A three-year programme priced today in INR, delivered by a team in Poland, billed in EUR, with salary inflation across the period.

Handling. Rate cards are effective-dated. Store the FX rate used on the estimate rather than looking it up live at read time. Support an annual escalation percentage applied per year of the schedule. Make the reporting currency an explicit, displayed choice.

11.16 Solo-builder burnout Severity: medium

Problem. Twenty-two weeks is five months. Motivation fails around week 7, right in the middle of the scheduling engine, which is the least visually rewarding work in the project.

Handling. Deliberately sequence a visible win after every hard block — the dashboard (Phase 5) directly follows the scheduler (Phase 4) for exactly this reason. Deploy to production from week 3 so progress is always visible on a real URL. Recruit design partners in Phase 0, not Phase 10; external expectation is the most reliable motivator available.

12. Non-functional requirements

Area	Target
Performance	p95 API latency < 300 ms (non-compute). Quick estimate < 400 ms. Full simulation < 2 s. Dashboard first contentful paint < 1.5 s on 4G.
Scale (v1)	500 organisations, 50 concurrent estimate computations, 100k stored estimate versions without query degradation.
Availability	99.5% for v1 (about 3.6 h/month). Compute service degradation must not take down read paths.
Security	OWASP Top 10 reviewed; argon2id password hashing; refresh-token rotation with reuse detection; TLS 1.3 only; secrets in a managed vault; RLS enforced; CSP and HSTS headers; dependency scanning in CI.
Privacy	GDPR and DPDP-aligned: export and delete endpoints, configurable retention, DPA template, region-pinned storage option.
Accessibility	WCAG 2.2 AA. Charts need accessible data-table equivalents — a colour-only Gantt fails audit.
Backups	Point-in-time recovery, 30-day window, restore drill executed before launch and quarterly after.
Observability	Structured JSON logs with trace IDs, RED metrics per endpoint, alerts on error rate, queue depth, and p95 latency.
Browser support	Last two versions of Chrome, Edge, Firefox, Safari. Dashboard is desktop-first; intake is responsive.

13. Infrastructure cost model

Stage	Setup	USD / month	INR / month
Development (weeks 1–13)	Local Docker, free tiers, one preview environment	$0 – 25	≈ 0 – 2,000
Beta (5–20 orgs)	Render/Railway small instances, Neon, Upstash, Sentry team, domain	$60 – 120	5,000 – 10,000
Early production (100 orgs)	2 API + 2 engine instances, managed Postgres 4 GB, Redis, S3, Sentry, Stripe fees	$250 – 450	21,000 – 38,000
Scale (500+ orgs)	AWS ECS multi-AZ, RDS with replica, ElastiCache, CloudFront, WAF, observability	$900 – 1,800	75,000 – 1,50,000

Excludes salaries, design, legal, and SOC 2 audit (a first SOC 2 Type II typically runs USD 20k–40k all-in including tooling and auditor).

14. Risk register

Risk	Prob.	Impact	Mitigation / trigger
Engine accuracy rejected by early users	High	Critical	Publish the methodology; validate against 10 real completed projects in Phase 0; ship calibration early
Phase 4 scheduling overruns	High	High	Timebox to 4 weeks; fall back to CPM without resource levelling for MVP
No design partners by week 6	Medium	High	Start outreach in Phase 0; if none by week 6, pause build and sell first
Scope creep into PM tooling	High	Medium	Section 2.3 reviewed at each planning session
Incumbent adds this feature	Medium	Medium	Calibration data moat; depth over breadth
Solo-builder capacity loss	Medium	High	Ship deployable increments; nothing takes longer than 3 weeks to reach a demo
Security incident exposing pricing data	Low	Critical	RLS, automated tenancy tests, pen test before commercial launch

15. Definition of done and success metrics

DEFINITION OF DONE, PER FEATURE

- Unit and integration tests written and passing; engine changes additionally have golden and property tests
- OpenAPI contract updated; shared types regenerated
- Error, empty, and loading states designed and implemented
- Accessibility checked with keyboard and screen reader
- Observability: at least one metric or log line that would reveal it failing in production
- Deployed to staging and exercised by a real user flow

PRODUCT METRICS TO INSTRUMENT FROM DAY ONE

Metric	Target at launch + 3 months
Wizard completion rate (start → locked estimate)	> 55%
Time to first completed estimate	< 12 minutes
Estimates per active org per month	> 4
Export rate (locked estimate → PDF downloaded)	> 70%
Orgs that submit actuals on a closed project	> 25% (this is the moat metric)
Week-4 retention of activated orgs	> 40%

16. Glossary

Term	Meaning
COCOMO II	Constructive Cost Model, 2nd generation — a parametric software effort estimation model
Function Point (FP)	A size measure based on delivered functionality rather than lines of code
KSLOC	Thousands of source lines of code
Person-month (PM)	One person working one month; a unit of effort, never of time
PERT	Three-point estimation: (optimistic + 4 x likely + pessimistic) / 6
Monte Carlo	Repeated random sampling to produce an outcome distribution
P50 / P80 / P95	Percentiles — P80 means 80% of simulated outcomes fall at or below this value
WBS	Work Breakdown Structure — hierarchical decomposition of project work
CPM	Critical Path Method — the longest dependent chain, which sets minimum duration
Resource levelling	Adjusting the schedule to respect finite people availability
Rayleigh curve	The characteristic ramp-peak-taper shape of project staffing over time
EVM	Earned Value Management — measuring progress in budget terms (PV, EV, AC)
SPI / CPI	Schedule and Cost Performance Index; below 1.0 means behind or over
EAC	Estimate At Completion — forecast total cost given performance to date
Contingency reserve	Budget for identified risks, typically P80 minus P50
RLS	Row-Level Security — database-enforced per-tenant data isolation
BFF	Backend For Frontend — a server layer tailored to one client
ADR	Architecture Decision Record — a short document capturing one decision and its rationale

> **Immediate next three actions.**
> 1. Interview eight people who currently produce estimates by hand; ask what they do today and what breaks. Do this before writing code.
> 2. Hand-build the Section 4 worked example in a spreadsheet. If the numbers do not feel right to an experienced practitioner, fix the model on paper — it is a thousand times cheaper there.
> 3. Scaffold the monorepo and deploy an empty authenticated app to a live URL by the end of week 3.

Estimation methodology references: Boehm et al., *Software Cost Estimation with COCOMO II*; McConnell, *Software Estimation: Demystifying the Black Art*; PMI, *Practice Standard for Project Estimating*; IFPUG Function Point Counting Practices Manual. Figures in this document are planning baselines, not commitments.